

Delete and Earn :

```
class Solution {
    public int deleteAndEarn(int[] nums) {
        int[] count = new int[10001];
        for(int x : nums) {
            count[x]++;
        }
        int earn1 = 0;
        int earn2 = 0;

        for(int x = 1; x <= 10000; x++) {
            int curEarn = x * count[x];
            if(count[x] == 0) {
                continue;
            }
            if(x > 1 && count[x - 1] > 0) {
                int temp = earn2;
                earn2 = Math.max(curEarn + earn1, earn2);
                earn1 = temp;
            }
            else {
                int temp = earn2;
                earn2 = curEarn + earn2;
                earn1 = temp;
            }
        }
        return earn2;
    }
}
```

Partition Equal Subset Sum :

```
class Solution {
    public boolean canPartition(int[] nums) {
        int n = nums.length;
        int sum = 0;
        for(int ele : nums) {
            sum += ele;
        }
    }
}
```

```

        if(sum % 2 != 0) return false;
        int target = sum / 2;

        boolean[] dp = new boolean[target + 1];
        dp[0] = true;
        for(int i = 0; i < n; i++) {
            for(int j = target; j >= nums[i]; j--) {
                dp[j] = dp[j] || dp[j - nums[i]];
            }
        }

        return dp[target];
    }
}

```

Target Sum :

```

class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int n = nums.length;
        int total = 0;
        for (int num : nums) {
            total += num;
        }
        if ((total - target) % 2 != 0) return 0;
        if (target > total || target < -total) return 0;

        int k = (total - target) / 2;
        int[][] dp = new int[n][k + 1];
        dp[0][0] = 1;

        if(nums[0] <= k)
            dp[0][nums[0]] = 1;

        if(nums[0] == 0)
            dp[0][0] = 2;

        for(int i = 1; i < n; i++) {
            for(int j = 0; j <= k; j++) {
                int notPick = dp[i - 1][j];
                int pick = 0;
            }
        }
    }
}

```

```

        if (nums[i] <= j) {
            pick = dp[i - 1][j - nums[i]];
        }
        dp[i][j] = pick + notPick;
    }
}
return dp[n - 1][k];
}
}

```

Minimum Cost For Tickets :

```

class Solution {
    public int mincostTickets(int[] days, int[] costs) {
        int n = days.length, m = costs.length;
        int[] dp = new int[n];
        Arrays.fill(dp, -1);

        return solve(days, costs, 0, 0, dp);
    }
    public int solve(int[] days, int[] costs, int i, int a, int[] dp) {
        if(i == days.length) {
            return 0;
        }
        if(days[i] <= a) {
            return solve(days, costs, i + 1, a, dp);
        }
        if(dp[i] != -1) return dp[i];

        int days1 = costs[0] + solve(days, costs, i + 1, days[i], dp);
        int days7 = costs[1] + solve(days, costs, i + 1, days[i] + 6, dp);
        int days30 = costs[2] + solve(days, costs, i + 1, days[i] + 29, dp);

        return dp[i] = Math.min(days1, Math.min(days7, days30));
    }
}

```

A. Boredom :

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] count = new int[100001];
        for (int i = 0; i < n; i++) {
            int x = sc.nextInt();
            count[x]++;
        }

        long[] dp = new long[100001];
        dp[1] = count[1];
        for (int i = 2; i <= 100000; i++) {
            dp[i] = Math.max(
                dp[i - 1],
                dp[i - 2] + (long)i * count[i]
            );
        }
        System.out.println(dp[100000]);
    }
}
```

C. Basketball Exercise

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long[] a = new long[n];
        long[] b = new long[n];

        for (int i = 0; i < n; i++) {
```

```

        a[i] = sc.nextLong();
    }
    for (int i = 0; i < n; i++) {
        b[i] = sc.nextLong();
    }

    long[] dp1 = new long[n];
    long[] dp2 = new long[n];
    dp1[0] = a[0];
    dp2[0] = b[0];

    for (int i = 1; i < n; i++) {
        dp1[i] = Math.max(dp1[i - 1], dp2[i - 1] + a[i]);
        dp2[i] = Math.max(dp2[i - 1], dp1[i - 1] + b[i]);
    }
    System.out.println(Math.max(dp1[n - 1], dp2[n - 1]));
}
}

```

B. Petr and a Combination Lock

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            a[i] = sc.nextInt();
        }
        for (int mask = 0; mask < (1 << n); mask++) {
            int sum = 0;
            for (int i = 0; i < n; i++) {
                if ((mask & (1 << i)) != 0) {
                    sum += a[i];
                } else {
                    sum -= a[i];
                }
            }
        }
    }
}

```

```
    }  
  }  
  if (sum % 360 == 0) {  
    System.out.println("YES");  
    return;  
  }  
}  
System.out.println("NO");  
}  
}
```